We also sanitised uploaded filenames and filtered file types to prevent users from uploading scripts disguised as documents.

Secure file uploads: File uploads are handled by Multer, with custom logic layered on top.

- Allowed extensions: *.pdf, .jpg, .png, .docx*
- Max file size: configurable per route
- Uploaded files are renamed using UUIDs and stored in a separate location from public assets.

While we haven't integrated a malware scanner like ClamAV yet, we have kept that option open for production environments.

We focused on practical, real-world security strategies that would hold up under moderate load and risk. JWT + RBAC gave us a strong baseline, while additional protections like input sanitisation and password hashing covered the essentials. And because security is never "done," we've kept the system modular enough to add more protections like rate limiting or two-factor auth, if and when the platform scales.

Frontend integration and real-time communication

Frontend development in SereneCare wasn't just about building clean pages -- it was about ensuring that everything connected smoothly and securely with the backend, while responding in real-time to events like appointment changes or group activity. We wanted the experience to feel live and responsive, especially for patients and doctors dealing with time-sensitive interactions.

Talking to the backend: Axios + interceptors: We used Axios across the frontend to handle all API communication. Instead of writing repetitive logic in every component, we created a shared instance that:

- Automatically attached the user's JWT token to every request
- Redirected to the login page if the token expired or was invalid
- Logged or displayed errors using toast notifications

This setup kept our API calls clean and centralised, and made it much easier to manage auth across multiple views.

Role-aware UI with React Context: To make sure each user only saw what they were supposed to, we used React Context to store global session data like:

- Logged-in user's role (patient, doctor, admin)
- Token status
- User profile info

This allowed us to dynamically load the correct dashboard on login without writing separate routes or pages for every role. It also helped us show/hide UI components like buttons or menus based on permissions.

Toasts, routing, and feedback loops: We integrated:

- React Hot Toast for clean, lightweight user notifications (e.g., 'Appointment request sent', 'Session expired')
- React Router for protected route handling, including role-based redirects
- UI loaders and disabled states to avoid double-submission issues on slower networks

These small details added up to a smoother experience for both patients and doctors using the app on a mobile or desktop.

Real-time features using WebSockets: One of the things we're proud of is how we added real-time responsiveness without over-complicating things. We used Socket.IO to open a WebSocket connection as soon as the user logs in. This allowed us to push updates to the UI for things like:

- Appointment status changes (e.g., confirmed, rescheduled)
- Support group messages
- New health documents uploaded by doctors

Each major component subscribed to specific events using the WebSocket channel. For example, when a doctor updated an appointment, the patient dashboard would receive an update within a second and no polling was required.

Figure 6 shows how the React frontend communicates with backend modules over HTTP (via Axios) and WebSockets for real-time interaction. React Context keeps global state consistent across views.

Initially, we considered using polling for live data, but it quickly became clear that it would strain both the client and server unnecessarily. WebSockets solved that elegantly. The combo of JWT for security, Axios for clean API calls, and React Context for session handling gave us a frontend that felt dynamic without being bloated.

Future enhancements

While SereneCare already covers the core features needed for remote healthcare management, we've mapped out several upgrades that we'd like to implement in the near future. These ideas came up naturally during testing and feedback, especially when we imagined the platform being used in more dynamic or real-world clinical settings.

Video consultations: Right now, all consultations happen through scheduling and messaging. One of our next big goals is to integrate video calling directly into the platform. We're currently exploring options like WebRTC for peer-to-peer connections and potentially using third-party APIs that support encryption and compliance with healthcare data standards.

Smart health recommendations: We also want to experiment with machine learning models that can analyse patient interactions or uploaded documents to offer basic health tips or reminders. For example, a patient frequently browsing mental health resources might get a prompt suggesting nearby counselling services or articles written by professionals.

To do this responsibly, we'll need to be careful about data handling, and